Comprehensive Development and Verification Strategy for Custom UAV Guidance, Navigation, and Control Systems

I. Executive Summary and Project Foundations (Concept & Requirements - Phase A/B)

The development of custom Guidance, Navigation, and Control (GNC) solutions for Unmanned Aerial Systems (UAS), particularly in industrial and heavy-lift applications, mandates a specialized approach to state estimation. Standard commercial off-the-shelf autopilots often lack the performance, integrity, or adaptability required for integrating novel sensor suites or handling specific vehicle dynamics, such as those associated with high-inertia maneuvers.[1] The development of a custom algorithm—referred to herein as the specialized state estimator—is essential for achieving requisite navigation accuracy, integrity, and continuity. This specialized solution is typically implemented as an Extended Kalman Filter (EKF) or an advanced hybrid fusion architecture, such as one integrating a Gated Recurrent Unit (GRU) with an EKF in a three-dimensional scenario to enhance robustness.[3]

To manage the complexity and risk associated with aerospace software development, project governance must adhere to a structured systems development lifecycle model. The V-Model is the recommended standard, graphically representing the rigorous decomposition of requirements on the left side, followed by the systematic integration and validation of the system on the right side.[4] This framework ensures high-level user needs (Validation: determining if the right product is being built) are traceable down to the low-level technical specifications and verified against those specifications (Verification: confirming the product is built correctly).[4]

1.2 Research and Foundational Study Resources

A robust implementation requires a deep theoretical foundation in control engineering and non-linear dynamics. Required study areas must encompass the core concepts of GNC, including the derivation of Nonlinear 6 Degree-of-Freedom (DOF) Equations of Motion, the principles governing Strapdown Inertial Navigation Systems (INS), and a thorough understanding of Navigation Reference Frames.

The literature review should prioritize research dedicated to optimizing filtering techniques for UAVs. This includes studies focusing on hybrid fusion architectures, such as the integration of GRU and EKF, which can improve estimator performance and reliability.[3] Furthermore, given the unique dynamics of industrial and heavy-lift platforms, it is crucial to consult research on observability analysis and simultaneous parameter identification utilizing the EKF framework.[6] This specialized area of research addresses the challenges posed by high-inertia systems, where accurately characterizing the vehicle's dynamic response is essential for stable navigation.

The necessity for a specialized GNC solution suggests that standard commercial EKF implementations, such as those found in open-source autopilots, are often insufficient due to difficulties in compensating for specific flight characteristics, such as high-inertia dynamics.[1] If generic EKF parameters, particularly those related to process noise, are used, they will fail to accurately account for the slow response times characteristic of heavy-lift platforms.[2] This inaccuracy often manifests as excessive position wander during high-precision flight modes like loiter.[1] Consequently, the successful creation of a custom GNC algorithm must incorporate or be preceded by a formal model identification process [6] to ensure the underlying dynamic lmodel accurately reflects the vehicle's unique characteristics.

Table 1.2.3 Foundational GNC Research and Study Resources

Resource Type	**Topic Coverage**	**Strategic Importance**
Textbooks (e.g., AIAA Course Material)	6 DOF Dynamics, Strapdown INS, Guidance Laws	Establishes the mathematical framework for the GNC state and process model [5]
Technical Papers (e.g.,	Hybrid GRU-EKF Fusion	Provides advanced

MDPI)	Architectures	techniques for improving estimator robustness and accuracy [3]
Identification Papers (e.g., IEEE/MDPI)	EKF for Parameter Identification, Observability	Necessary for accurately tuning the filter state model for heavy-lift platforms [6]

II. Requirements Definition and Theoretical Design (Left Side of V - Phase B/C)

This phase involves translating the operational requirements into rigorous mathematical definitions, forming the core of the custom GNC software. This theoretical foundation dictates the success of all subsequent implementation and verification activities.

2.1 Theoretical GNC Algorithm Design

The state estimation process begins with the definition of the state vector (). For a typical multi-rotor system, this vector includes core navigation variables such as position, velocity, and attitude (often represented by quaternions), angular rates, and potentially augmented states like sensor biases or dynamic vehicle parameters if simultaneous online identification is performed.[6]

Since the dynamics of an unmanned aerial vehicle (UAV) are inherently nonlinear, the Extended Kalman Filter (EKF) is selected as the appropriate framework. The EKF operates by approximating the nonlinear system function and the measurement function using a first-order Taylor series expansion linearized around the current state estimate.[8] The process function

 must be derived directly from the vehicle's 6 DOF equations of motion, which accounts for coupled attitude dynamics necessary for accurate prediction.[9]

The EKF structure involves two primary steps:

1. **Prediction Step:** The state and covariance matrix are propagated forward in time using the nonlinear dynamic model and the linearized process Jacobian, often referred to as the matrix.[10]
2. **Update Step:** The predicted state is corrected using external measurements (e.g., high-precision GPS, RTK data, or specialized vision systems). This correction relies on the linearized measurement Jacobian, or matrix.[10]

2.2 Jacobian and Covariance Management

The analytical derivation of the and matrices is perhaps the most mathematically intensive task in this phase, requiring calculation of the partial derivatives of the process and measurement functions with respect to every element in the state vector.[10]

Critical to the EKF's performance is the accurate tuning of the Process Noise Covariance matrix (). An inaccurate matrix, particularly in high-inertia systems, will prevent the filter from effectively correcting inertial drift.[1] The

 matrix is typically derived based on the Jacobian of the process function with respect to the process noise ().[10] This derivation requires characterizing the noise standard deviation of onboard gyroscope measurements () and introducing a tuning scaling factor () to adjust the overall weight of the process model uncertainty during convergence.[11]

The EKF relies fundamentally on a linearization point defined by the current state estimate.[8] If the filter's covariance propagation () is inaccurate due to poorly tuned parameters—a common pitfall when dealing with the high-inertia characteristics of heavy-lift platforms—the linearization point will drift significantly from the true vehicle state. This substantial deviation invalidates the first-order approximation, a condition that severely increases the risk of filter divergence, representing a critical failure mode. Consequently, the theoretical design must incorporate rigorous sensitivity analysis of both and the measurement noise covariance () matrices against anticipated environmental factors such as high vibration and magnetic field perturbations.

2.3 Sensor and Coordinate Frame Management

The GNC algorithm must operate within a meticulously managed hierarchy of coordinate

systems to fuse global and local data sources effectively. Three frames are critical:

1. **WGS-84 (World Geodetic System):** The standard frame used by GNSS receivers, defining location via Latitude, Longitude, and Altitude.[12]
2. **ECEF (Earth-Centered, Earth-Fixed):** A geocentric, non-inertial frame used for high-precision calculations, defined by its Z-axis aligned with the Earth's minor axis.[12]
3. **NED (North-East-Down):** The local tangent plane frame, which typically serves as the primary reference for the vehicle's control loops.[13]

Robust implementation necessitates verified software libraries for the non-linear transformation between NED coordinates and geodetic coordinates (LLA) based on the WGS-84 ellipsoid model.[13] This requirement for coordinate system transformation [12] indicates that the custom GNC algorithm must natively process data in a dynamically stable frame (such as NED) different from the raw sensor input (LLA/WGS-84). The inherent complexity and non-linearity of these transformations demand the use of high-quality, pre-validated libraries, such as those available in specialized C++ navigation packages [14], to ensure numerical stability and minimize computation errors within the real-time filtering process.

III. System Architecture and Hardware Specification (Phase C)

High-integrity GNC requires a specialized hardware architecture focusing on redundancy, precision, and dedicated computational resources.

3.1 Flight Vehicle Platform and Control Architecture

The baseline platform should be an industrial, modular, and field-ready system, such as the HLM Quadcopter with its 20 kg payload capacity.[2] Such platforms are designed to comply with aerospace standards, including ASTM F38 and FAA Part 108, which dictate safety and redundancy requirements.[2]

Safety-critical operations mandate the use of redundant flight controllers (FCs), exemplified by systems like CubePilot.[15] This redundancy is not a mere failsafe; it is foundational to safety compliance and ensuring the necessary continuity of operation during potential single-point

failures.[2]

A high-performance companion computer (SBC) is required to host the custom GNC algorithm. This architectural decision is vital: running the experimental, computationally demanding EKF on a dedicated processor isolates the custom code from the primary flight controller's critical execution loop. This minimizes risk and avoids destabilizing the certified flight firmware.

3.2 High-Precision Navigation Sensor Suite: RTK GNSS

Achieving the high-performance navigation required for industrial applications demands centimeter-level positioning accuracy, mandating the use of Real-Time Kinematic (RTK) Global Navigation Satellite System (GNSS) technology. The u-blox ZED-F9P module is the standard choice for this requirement, offering multi-band, multi-constellation GNSS support.[15] This hardware selection ensures superior signal reception and enhanced robustness against potential jamming or interference.[16]

Proper configuration of the RTK system using the u-center software is paramount for realizing its full potential.[17] The module must be configured to output essential data streams, such as UBX-RXM-RAWX and UBX-RXM-SFRBX for raw data logging, or NMEA-GGA for compatibility with various external systems.[17]

A critical configuration step specific to high-integrity applications involves disabling Satellite-Based Augmentation Systems (SBAS). This is a deliberate strategy to prevent potentially unstable position jumps that occur when an RTK fix is briefly lost and the system attempts to revert to the lower-integrity SBAS solution.[17] Such position jumps would introduce sharp, erroneous measurement innovations into the EKF, risking filter instability. By disabling SBAS, the system is forced to rely on inertial prediction during RTK dropout, maintaining the integrity and consistency required by the EKF measurement update step. Furthermore, achieving higher navigation rates (e.g.,

) may require optimizing constellation usage, potentially by disabling specific systems like Beidou, to guarantee stability and prevent processor overload.[17]

Table 3.3.4 Essential Hardware Components and Rationale

Component Category	Example Specification	Justification for High-Integrity GNC
Flight Controller (FC)	Industrial Redundant Autopilot (e.g., CubePilot)	Ensures physical redundancy, supports failover, required for safety compliance (e.g., FAA Part 108) [2]
Primary GNSS Receiver	u-blox ZED-F9P	Multi-band, multi-constellation RTK required for centimeter-level accuracy input to EKF [15]
Telemetry Link	LTE Module (e.g., SARA-R5)	Robust transmission of RTK correction data (NTRIP) and high-bandwidth mission control data
GNC Processor	Companion Computer (High-performance SBC)	Dedicated resource for running the custom "Own Thing," isolating computational load from the FC's critical loop

IV. Software Implementation and Integration (Base of V - Phase C/D)

2

The transition from theoretical mathematical model to embedded execution requires a disciplined software development approach that emphasizes automated verification and low-latency communication.

4.1 Development Environment and Autocoding Strategy

Model-Based Design (MBD) is the standard method for GNC software development. High-level simulation tools such as MATLAB/Simulink are used to create the GNC prototype for Functional Engineering Simulation (FES) and Model-In-the-Loop (MIL) testing.[19] This environment enables rapid design iteration and immediate functional verification of the control laws against a nonlinear plant model.

A crucial bridge between the theoretical design (Left Side of the V) and the embedded code (Right Side of the V) is Autocoding. The verified GNC Simulink model is automatically translated into optimized ANSI C-code suitable for embedded systems.[19] This process is critical for maintaining high integrity, as it minimizes the risk of human programming errors introducing numerical discrepancies or logic faults into the execution environment. This step serves as the initial, formal verification that the implementation aligns mathematically with the validated model.

4.2 Middleware Integration: MAVSDK and DDS

The custom GNC algorithm residing on the companion computer must seamlessly exchange high-rate data with the flight controller (FC). The choice of middleware depends on the required communication latency.

MAVSDK provides a comprehensive, high-level library for accessing telemetry data (e.g., IMU readings, raw GPS signals) and sending high-level commands (Action, Mission).[20] However, for high-bandwidth, low-latency, real-time exchange of the custom state estimate—the output of the "Own Thing"—the uXRCE-DDS framework is required.[21] In this architecture, the companion computer hosts the DDS agent, which acts as a proxy, while the FC runs the DDS client, enabling publishing and subscribing to defined topics on the DDS data space, sharing PX4 uORB topics.[21]

The choice between MAVSDK and DDS reflects a strategic decision regarding the GNC insertion point. If the custom EKF is merely providing high-level guidance or supplemental telemetry, MAVSDK suffices. However, if the algorithm is replacing or tightly augmenting the FC's internal state estimator, requiring millisecond-level position and velocity inputs to the attitude control loop, then the low-latency DDS protocol is essential to maintain real-time performance and control stability.[21]

4.3 Coding Constraints and Architecture

The physical implementation of the custom EKF must respect the constraints imposed by the interfacing middleware. MAVSDK specifically cautions developers: computationally heavy tasks, such as input/output (IO) operations, must not be executed inside callback functions, as this will stall the single thread used for user callbacks, severely impacting real-time data flow.[20]

To circumvent this, the custom EKF, which requires intensive matrix operations (multiplications, inversions, and Jacobian calculations), must be implemented within a dedicated, high-priority, isolated thread on the companion computer. The callback function is then strictly limited to quickly queuing input sensor data and retrieving the latest calculated state estimate, preventing stalls in the mission critical data pathways. Furthermore, validated C++ navigation libraries (e.g., Bolderflight) should be used for all mathematical routines, particularly for complex coordinate transformations (NED LLA), ensuring numerical efficiency and correctness within the embedded environment.[13]

V. Verification, Validation, and Deployment (Right Side of V - Phase D/E)

The verification and validation (V&V) phase follows an integrated, incremental DDVV (Design, Development, Validation, and Verification) chain to confirm that the GNC system is built correctly and satisfies operational requirements.[19]

5.1 Verification Phase: Building it Right

Verification confirms that the GNC Application Software (ASW) meets its low-level technical specifications.[4] The progression from model to integrated hardware guarantees integrity:

1. **Model-In-the-Loop (MIL) Testing:** The GNC prototype's functional logic is tested in a full simulation environment using high-level tools like Matlab/Simulink.[19] This stage verifies the mathematical rigor of the control laws against ideal plant models.

2. **Software-In-the-Loop (SIL) Testing:** The automatically generated ANSI C-code [19] is executed in a simulated environment and compared against the results of the verified MIL model. This ensures that the autocoding process did not introduce numerical errors.
3. **Processor-In-the-Loop (PIL) Testing:** The SIL-verified GNC ASW is embedded onto a representative target processor (the companion computer) and tested in a real-time, simulated environment.[9] This stage critically assesses the computational load, execution latency, and resource utilization, guaranteeing that the intensive EKF calculations—which include matrix inversion for covariance updates—do not breach the necessary real-time execution deadlines. If this stage reveals excessive latency, the project must loop back to the Design phase to potentially simplify the filter architecture or state vector size.[9]

5.2 EKF Health Monitoring and Validation Metrics

Effective filter tuning requires meticulous management of the covariance parameters based on expected operational dynamics.

The **Process Noise Covariance ()** must be precisely tuned for the high-inertia platform. If is set too large, the filter places too much confidence in the inertial sensors, causing the estimated position to wander slowly because inertial calculation errors are not sufficiently corrected by GPS measurements.[1] Conversely, setting

 too small causes the filter to rely too heavily on measurements, making it overly susceptible to sensor noise.

To maintain measurement integrity, the EKF employs an **Innovation Consistency Check**, managed by parameters such as EKF_POS_GATE.[1] This scaling threshold is used to validate incoming GPS measurements against the state prediction. Decreasing this parameter increases the probability of rejecting legitimate measurements, while increasing it makes the filter vulnerable to incorporating anomalous or faulty sensor data, which could destabilize the state estimate.[1] Rigorous post-flight log analysis, including monitoring covariance propagation and analyzing measurement innovations against statistical health metrics (such as the Chi-square distribution), is mandatory to validate that the tuned parameters (

 and) are statistically sound under actual flight loads.[9]

5.3 Hardware-In-the-Loop (HIL) Testing

The final verification step before physical deployment is Hardware-In-the-Loop (HIL) testing. The actual flight controller and the custom GNC processor (running the PIL-verified code) are connected to a simulator that injects realistic synthetic sensor inputs (e.g., simulated IMU, RTK GPS, Baro data) and receives control outputs.[9] The purpose of HIL is to verify the total system integration, including the efficacy of the MAVSDK/DDS interface protocols, hardware timing synchronization, and the GNC algorithm's response to dynamic maneuvers and simulated sensor faults, such as GPS signal dropout or the injection of IMU bias errors.

Table 5.3.3 The DDVV Chain: V&V Stages and Deliverables

V&V Stage	**Objective**	**Artifact Verified**	**Key Output/Deliverable**
MIL (Model-In-the-Loop)	Verify functional correctness of the mathematical model.	GNC Simulink/Matlab Model	Model Verification Report (MVR) [19]
SIL (Software-In-the-Loop)	Verify the execution integrity of the generated C-code.	Autocoded ANSI C-code	Code Integrity and Coverage Report [19]
PIL (Processor-In-the-Loop)	Verify real-time performance on target CPU.	GNC ASW executable on Companion Computer	Real-Time Latency and Computational Load Metrics [9]
HIL (Hardware-In-the-Loop)	Validate hardware integration and dynamic system response.	Flight Controller + GNC Processor + Sensor Interface	System Integration Test Report (SITR) [19]

VI. Conclusion and Strategic Recommendations

The development of a custom GNC system for industrial UAS platforms must be structured as a high-integrity systems engineering endeavor, strictly governed by the V-Model lifecycle. This approach ensures meticulous traceability from validation of user requirements to final hardware verification.

The critical technical hurdles reside in the theoretical design (Phase II). The high mass and inertia characteristic of heavy-lift platforms mandate that the EKF process model must be accurately identified and tuned. Failure to accurately characterize the Process Noise Covariance () will cause the filter's linearization point to drift, leading to potential instability or filter divergence in flight.[1] The mandatory architectural solution involves utilizing industrial-grade, redundant hardware, specifically the multi-band RTK GNSS (u-blox ZED-F9P) [15], configured to prioritize data integrity (e.g., disabling SBAS).[17]

Furthermore, the software architecture must utilize Model-Based Design and Autocoding to bridge the gap between mathematical rigor and embedded execution.[19] For the real-time injection of the custom state estimate into the flight control loop, the low-latency uXRCE-DDS protocol is necessary, ensuring the millisecond timing constraints of high-rate control systems are met.[21] Ultimately, system readiness is confirmed only by the successful completion of the integrated DDVV chain, culminating in HIL testing, which verifies the GNC loop closure under simulated dynamic and failure conditions.[9] Any failures detected during PIL testing, particularly those related to the EKF's computational load, necessitate an iterative return to the theoretical design phase before proceeding to flight.

Works cited

1. Extended Kalman Filter Navigation Overview and Tuning — Dev documentation - ArduPilot, accessed on October 5, 2025, https://ardupilot.org/dev/docs/extended-kalman-filter.html
2. Compare Heavy Lift Drones - Aero Systems West, accessed on October 5, 2025, https://aerosystemswest.com/compare-heavy-lift-drones/
3. Resilient Multi-Sensor UAV Navigation with a Hybrid Federated Fusion Architecture - MDPI, accessed on October 5, 2025, https://www.mdpi.com/1424-8220/24/3/981
4. V-model - Wikipedia, accessed on October 5, 2025, https://en.wikipedia.org/wiki/V-model
5. Flight Vehicle Guidance Navigation and Control Systems (GNC): Analysis and

Design - AIAA - Shaping the future of aerospace, accessed on October 5, 2025, https://aiaa.org/courses/flight-vehicle-guidance-navigation-and-control-systems-gnc-analysis-and-design/
6. EKF-Based Parameter Identification of Multi-Rotor Unmanned Aerial VehiclesModels - PMC, accessed on October 5, 2025, https://pmc.ncbi.nlm.nih.gov/articles/PMC6806097/
7. EKF-Based Parameter Identification of Multi-Rotor Unmanned Aerial VehiclesModels - MDPI, accessed on October 5, 2025, https://www.mdpi.com/1424-8220/19/19/4174
8. Physics-Informed Neural Networks for Unmanned Aerial Vehicle System Estimation - MDPI, accessed on October 5, 2025, https://www.mdpi.com/2504-446X/8/12/716
9. Application of Extended Kalman Filter Towards UAV Identification - ResearchGate, accessed on October 5, 2025, https://www.researchgate.net/publication/227029817_Application_of_Extended_Kalman_Filter_Towards_UAV_Identification
10. Implementation of an Extended Kalman Filter Using Inertial Sensor Data for UAVs During GPS Denied Applications - ODU Digital Commons, accessed on October 5, 2025, https://digitalcommons.odu.edu/context/mae_etds/article/1347/viewcontent/Seliquini_implementation_of_an_extended_Kalman_filter.pdf
11. Bayesian Optimization for Fine-Tuning EKF Parameters in UAV Attitude and Heading Reference System Estimation - MDPI, accessed on October 5, 2025, https://www.mdpi.com/2226-4310/10/12/1023
12. science conops for application of sport mission data to study large (~1000km) ionospheric plasma depletions - NASA Technical Reports Server (NTRS), accessed on October 5, 2025, https://ntrs.nasa.gov/api/citations/20220011194/downloads/SSC22-Final.docx.pdf
13. ned2ecef - Transform local north-east-down coordinates to geocentric Earth-centered Earth-fixed - MATLAB - MathWorks, accessed on October 5, 2025, https://www.mathworks.com/help/map/ref/ned2ecef.html
14. bolderflight/navigation: Navigation filters, transforms, and utilities - GitHub, accessed on October 5, 2025, https://github.com/bolderflight/navigation
15. RTK GNSS technology brings high precision to drone navigation | - u-blox, accessed on October 5, 2025, https://www.u-blox.com/en/casestudies/high-precision-drone-navigation
16. ZED-F9P module - u-blox, accessed on October 5, 2025, https://www.u-blox.com/en/product/zed-f9p-module
17. How to configure u-blox ZED-F9P - ArduSimple, accessed on October 5, 2025, https://www.ardusimple.com/how-to-configure-ublox-zed-f9p/
18. u-blox ZED-F9P - Sparkfun, accessed on October 5, 2025, https://cdn.sparkfun.com/assets/learn_tutorials/8/5/6/ZED-F9P_UBX_NMEA_and_RTCM_protocols.pdf

19. Design, Development, Validation and Verification of GNC technologies - eucass, accessed on October 5, 2025, https://www.eucass.eu/doi/EUCASS2019-0038.pdf
20. MAVSDK Paradigms/Usage, accessed on October 5, 2025, https://mavsdk.mavlink.io/main/en/cpp/guide/general_usage.html
21. uXRCE-DDS (PX4-ROS 2/DDS Bridge) | PX4 Guide (main), accessed on October 5, 2025, https://docs.px4.io/main/en/middleware/uxrce_dds.html